COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	**CPE 010 - DATA STRUCTURE AND ALGORITHM**	**SECTION:**	CPE21S4
DATE PERFORMED:	10/16/26	**DATE SUBMITTED:**	10/16/26
NAME:	Del Mundo, Kenneth James c. Mangcucang, Rhianne L	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Activity NO.12.1 ALGORITHM STRATEGIES

1. PROCEDURE OUTPUT

STRATEGY	ALGORITHM	ANALYSIS
Recursion	**Factorial Calculation**	**MANGCUCANG:** **The function calls itself with a smaller input until it reaches the base case, which stops the recursive calls.** **DEL MUNDO:** The factorial calculation uses recursion by calling the same function with a smaller number until it reaches the base case. The base case stops the function from calling itself and allows the program to calculate the factorial result.
Brute Force	**Linear Search**	**MANGCUCANG:** **The algorithm checks each element sequentially until the target is found or all elements have been checked.** **DEL MUNDO:** Linear search checks each element in the list one by one until the target value is found or all elements have been checked. It is simple to use, but it can take more time when the list contains many elements.

Backtracking	**N-Queens Problem**	**MANGCUCANG:** **The algorithm builds a solution incrementally and abandons a path when it violates a constraint.** **DEL MUNDO:** The N-Queens problem uses backtracking to place queens on a chessboard without allowing them to attack each other. If a queen is placed in the wrong position, the algorithm goes back and tries another position until it finds a valid solution.
Greedy	**Dijkstra's Shortest Path Algorithm**	**MANGCUCANG:** **The algorithm repeatedly selects the unvisited vertex with the smallest tentative distance.** **DEL MUNDO:** Dijkstra's algorithm finds the shortest path from a starting point to other points in a graph. It repeatedly selects the unvisited point with the smallest known distance and updates the distances to its neighboring points.
Divide and Conquer	**Merge Sort**	**MANGCUCANG:** **The algorithm divides the array into smaller parts, sorts them recursively, and merges the sorted parts** **DEL MUNDO:** Dijkstra's algorithm finds the shortest path from a starting point to other points in a graph. It repeatedly

		selects the unvisited point with the smallest known distance and updates the distances to its neighboring points.

Table 12-1. Algorithmic Strategies and Examples

SCREENSHOT:

CODE:

```
#include <iostream>
#include <algorithm>
using namespace std;

int *memo;

int getMinSteps(int n)
{
    if (n == 1)
        return 0;

    if (memo[n] != -1)
        return memo[n];

    int r = 1 + getMinSteps(n - 1);

    if (n % 2 == 0)
        r = min(r, 1 + getMinSteps(n / 2));

    if (n % 3 == 0)
        r = min(r, 1 + getMinSteps(n / 3));

    memo[n] = r;

    return r;
}

int main()
{
    int n;

    cout << "Enter a positive integer: ";
    cin >> n;

    if (n < 1)
    {
        cout << "Please enter a positive integer.\n";
        return 0;
    }

    memo = new int[n + 1];

    for (int i = 0; i <= n; i++)
        memo[i] = -1;

    memo[1] = 0;

    cout << "Minimum number of steps: "
         << getMinSteps(n) << endl;

    delete[] memo;

    return 0;
}
```

OUTPUT:

```
Enter a positive integer: 10
Minimum number of steps: 3
```

ANALYSIS:
DELMUNDO: Memoization uses recursion to solve the Minimum Steps to One problem. It finds the minimum number of operations needed to reduce a positive integer to 1. The program considers subtracting 1, dividing by 2 if divisible by 2, or dividing by 3 if divisible by 3. It stores the previous results to avoid repeating calculations, making the program more efficient.
MANGCUCANG: The memoization and bottom-up dynamic programming approaches solve the Minimum Steps to One problem by finding the minimum number of operations needed to reduce a positive integer to 1. Both approaches consider subtracting 1, dividing by 2 if the number is divisible by 2, and dividing by 3 if the number is divisible by 3. The min() function compares the possible operations and selects the one that leads to the fewest total steps.

Table 12-2. Memoization Implementation

SCREENSHOT:

CODE:

```cpp
#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;

int getMinSteps(int n)
{
    vector<int> dp(n + 1);

    dp[1] = 0;

    for (int i = 2; i <= n; i++)
    {
        dp[i] = 1 + dp[i - 1];

        if (i % 2 == 0)
            dp[i] = min(dp[i], 1 + dp[i / 2]);

        if (i % 3 == 0)
            dp[i] = min(dp[i], 1 + dp[i / 3]);
    }

    return dp[n];
}

int main()
{
    int n;

    cout << "Enter a positive integer: ";
    cin >> n;
```

```
    if (n < 1)
    {
        cout << "Please enter a positive integer.\n";
        return 0;
    }

    cout << "Minimum number of steps: "
         << getMinSteps(n) << endl;

    return 0;
}
```

OUTPUT:

```
Enter a positive integer: 10
Minimum number of steps: 3
```

ANALYSIS:

DELMUNDO: Bottom-up dynamic programming solves the Minimum Steps to One problem using a loop instead of recursion. It starts from 1 and calculates the minimum steps needed for each number until it reaches the given input. The program checks subtracting 1 and dividing by 2 or 3 when possible. It stores the results in an array and returns the minimum number of steps needed to reach 1.

MANGCUCANG:
Bottom-up dynamic programming uses an iterative approach instead of recursion. It creates a dp array and sets dp[1] to 0 because no steps are needed to reach 1. A loop then calculates the minimum number of steps for each number, starting from 2 and continuing up to the input number. For every number, the program considers subtracting 1 and dividing by 2 or 3 when possible, storing the minimum result in the dp array. Finally, it returns dp[n] as the answer.

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Create C++ code to implement various algorithmic strategies & ILO C: Create C++ code to solve problems using learned algorithmic strategies

PSEUDOCODE:

```
START
   INPUT matrix, rows, columns, targetCost

   FUNCTION countPaths(row, col, remainingCost)
      IF row >= rows OR col >= columns
         RETURN 0

      remainingCost = remainingCost - matrix[row][col]

      IF row = rows - 1 AND col = columns - 1
         IF remainingCost = 0
            RETURN 1
         ELSE
            RETURN 0

      RETURN countPaths(row, col + 1, remainingCost)
          + countPaths(row + 1, col, remainingCost)

   OUTPUT countPaths(0, 0, targetCost)
END
```

Path 1:

Right ⟶ Right ⟶ Down ⟶ Down ⟶ Down ⟶ Right

$4 + 7 + 1 + 3 + 1 + 7 + 3 = 26$

This path does not qualify because its cost is 26.

Path 2:

Down ⟶ Down ⟶ Right ⟶ Right ⟶ Down ⟶ Right

$4 + 6 + 3 + 8 + 1 + 7 + 3 = 32$

This path also does not qualify because its cost is 32.

The algorithm must examine the possible paths and count only those whose total cost is exactly 25.

CODE:

```cpp
#include <iostream>
#include <vector>
using namespace std;


int rows = 4, cols = 4;
int matrix[4][4] = {
    {4, 7, 1, 6},
    {6, 7, 3, 9},
    {3, 8, 1, 2},
    {7, 1, 7, 3}
};


int targetCost = 25;
int memo[4][4][26];


int countPaths(int row, int col, int remainingCost) {
    // Check if the position is outside the matrix
    if (row >= rows || col >= cols)
        return 0;


    // Subtract the current cell's cost
    remainingCost -= matrix[row][col];


    // Reject paths that exceed the target cost
    if (remainingCost < 0)
        return 0;


    // Check if the destination is reached
    if (row == rows - 1 && col == cols - 1) {
        return remainingCost == 0 ? 1 : 0;
    }


    // Return the saved result if already calculated
    if (memo[row][col][remainingCost] != -1)
        return memo[row][col][remainingCost];


    // Move right or down
    int right = countPaths(row, col + 1, remainingCost);
    int down = countPaths(row + 1, col, remainingCost);
```

```
    // Save and return the total number of paths
    memo[row][col][remainingCost] = right + down;


    return memo[row][col][remainingCost];
}


int main() {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            for (int k = 0; k <= targetCost; k++) {
                memo[i][j][k] = -1;
            }
        }
    }


    int result = countPaths(0, 0, targetCost);


    cout << "Matrix:" << endl;
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }


    cout << "\nTarget Cost: " << targetCost << endl;
    cout << "Number of paths: " << result << endl;


    return 0;
}
```

OUTPUT:

```
Matrix:
4 7 1 6
6 7 3 9
3 8 1 2
7 1 7 3

Target Cost: 25
Number of paths: 0
```

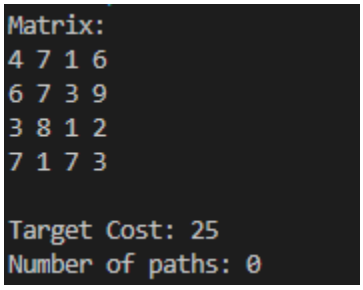
ANALYSIS:

DEL MUNDO:

The program uses recursion to look for paths from the starting cell to the destination. At every cell the program tries moving and moving down. The program subtracts the cost of each visited cell from the remaining target cost. If the remaining cost goes below zero the program rejects that path. When the program reaches the destination the program counts the path only if the remaining cost is zero.Memoization stores past results so the program does not redo the calculations. This makes the program faster, than using recursion alone.

MANGCUCANG:

The program uses recursion to find possible paths from the starting cell to the destination by moving right or down. It subtracts the cost of each visited cell from the target cost of 25. When a path reaches the destination, the program counts it only if the total cost is exactly 25. Memoization helps improve efficiency by storing previous results and avoiding repeated calculations.



CONCLUSION:

DELMUNDO:

In conclusion, this activity helped me understand different algorithmic strategies and how they solve problems. Recursion, brute force, backtracking, greedy, and divide and conquer each have different ways of finding solutions. I also learned that memoization and bottom-up dynamic programming can find the minimum steps more efficiently by storing and reusing results. Overall, this activity improved my understanding of algorithms and how to apply them in C++ programming.

MANGCUCANG:

In this activity, I learned how different programming techniques and algorithmic strategies solve problems efficiently. I understood how recursion, brute force, backtracking, greedy algorithms, and dynamic programming work in different situations. I also learned that memoization and bottom-up dynamic programming help reduce repeated calculations and find the minimum number of steps. Through the path-finding problem, I understood how recursion and memoization can be used to check possible paths and count those that meet a specific cost.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.